AutoSecAI: Multi-Agent Pull Request Analysis for Security, Vulnerability Risk and Code Quality

CHEPURI RISHIVARDHAN
Roll No. 23K91A0553
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology (Autonomous)
[CITY]
[EMAIL]

BODA SANDEEP
Roll No. 23K91A0539
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology (Autonomous)
[CITY]
[EMAIL]

DHARAVATH SEETHARAM
Roll No. 23K91A0562
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology (Autonomous)
[CITY]
[EMAIL]

NARISIMHA REDDY
Roll No. 24K95A0501
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology (Autonomous)
[CITY]
[EMAIL]

P. KISHORE
[DESIGNATION], Project Guide
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology (Autonomous)
[CITY]
[EMAIL]

***Abstract*—**Pull requests are the main entry point for security flaws, yet manual review is slow, depends on the reviewer's security expertise, and static analysers bury developers in warnings. This paper presents AutoSecAI, a pull-request analyser in which seven cooperating agents process a change: ingestion, AST and taint-based static analysis, a machine-learning risk agent that scores every changed function from 14 source-code metrics, a security reasoner, a remediation agent that verifies each patch by re-scanning it, a quality reviewer and a judge that issues a risk score and a merge verdict. A codebase assistant answers questions about a repository with cited code. The risk model reproduces a published metrics-based pipeline on Python; because the original C datasets are unavailable it is trained on a generated corpus of 6,000 functions. On 1,500 held-out functions XGBoost reaches F1 0.851 (mean 0.842 over five corpora), the rule engine 0.800, and flagging by either detector 0.891. The built-in demo pull request is reviewed in a median of 0.375 s with every patch verified. The data are generated, no labelled real-PR benchmark exists, and the optional language-model layer was not measured.

***Index Terms*—**automated code review, codebase intelligence, multi-agent systems, pull requests, source-code metrics, static analysis, vulnerability prediction.

I. INTRODUCTION

Every change that reaches a shared code base arrives as a pull request, so each one is a possible entry point for an injection flaw, a leaked secret or a weak cryptographic call. Review is done by people: it is slow, it varies with the reviewer's security knowledge, and it lacks the context of a large repository when judging a small diff. Static analysers catch known patterns but return many low-value warnings with little explanation, and source-code metrics, which correlate with vulnerability, are used offline on whole projects rather than inside the review. Language models can explain a change in context, but a single general reviewer is hard to audit.

AutoSecAI answers this with specialised agents, each with one narrow job and an auditable trace. Static rules supply precise, explainable findings; a metric model ranks functions by how much their structure resembles that of vulnerable code and corroborates or questions the rules; a remediation agent proposes a fix and proves that the rule no longer fires; and a judge turns the evidence into one decision a reviewer can act on. Nothing is merged, posted or changed automatically: the system advises and the reviewer decides.

The base paper [1] predicts vulnerable C/C++ functions from 14 code metrics, and recent work moves towards model-based pull-request review [2]. The gap is applying the metric signal inside the live pull-request workflow together with rule-based analysis, explanation and a checked fix. This paper presents AutoSecAI. Its contributions are:

1) a seven-agent pull-request pipeline that combines an AST and taint rule engine (30 security rules mapped to CWE and OWASP) with a metric-based risk model and issues a risk score and a merge verdict;
2) a Python adaptation of the 14 metrics of the base paper [1] and a reproduction of its selection, balancing and nine-model pipeline, with three added taint metrics;
3) patches that are verified by re-scanning, exports (Markdown, SARIF, JSON, CSV), and a codebase assistant that answers repository questions with cited code;
4) an honest evaluation: rule engine against ML on the same held-out functions, five corpus seeds, latency and automated tests.

Section II reviews related work, Section III describes the method, Section IV reports the results and Section V concludes.

A. System Hardware

All measurements were taken on one desktop: Intel Core i5-12500 (12 threads), 15.7 GB of memory, no discrete graphics card, Windows 11. Models are trained and run on the CPU.

B. Software Requirements

Python 3.11.9 with scikit-learn 1.5.0 [3], XGBoost 2.0.3 [4], pandas, NetworkX and the standard ast module; FastAPI with PyJWT, bcrypt and SQLite for the service; React 19, TypeScript, Vite and Tailwind CSS for the web interface; pytest and Playwright for tests. An Anthropic key is optional and unused in every measurement.

II. LITERATURE SURVEY

Base paper. Abbasi and Shameli-Sendi [1] study which source-code factors predict vulnerabilities and their types. Their data are 21,759 functions from ten C/C++ projects (SQLite, FreeType, LibTIFF, Libxslt, Binutils, FFmpeg, ImageMagick, OpenSC, rdesktop and OpenSSL), of which 275 (1.26 %) are vulnerable, with types taken from CVE and NVD. A fuzzy parser builds a code property graph from which 14 metrics are computed, among them cyclomatic complexity [5], caller-argument count, pointer arithmetic, nesting depth and control dependency. Seven feature-selection methods vote for the five most important metrics (M5, M6, M1, M9 and M11); the imbalance is treated with SmoteTomek [6], [7], [8], which yields 21,474 balanced samples; values are Min-Max normalised; and nine models are tuned by grid search and repeated k-fold validation. Random Forest [9] performs best: accuracy 0.96, precision 0.948, recall 0.974 and F1 0.961, and 0.972 accuracy for one vulnerability type (denial of service). The work is an offline function classifier for C/C++: it produces no explanation, no fix and no integration with review.

Pull-request review. Sphinx [2] builds context-rich review comments by comparing pseudo-modified and merged code, evaluates them with a checklist of actionable verification points instead of BLEU or ROUGE, and trains a model with rule-based checklist rewards; it reports up to 40 % higher checklist coverage than baselines. Four further works are listed in our batch survey by title only: a CNN-LSTM hybrid for vulnerability detection [10], real-time AI code security auditing with remediation [11], a multi-modal framework with code simplification and meta-learning [12], and DcodeBERT for detection and repair [13]. We could not retrieve their text and state no result for them.

Learning-based vulnerability detection. Earlier work learns from abstract syntax trees [14], deep learning on code [15] and, for Python specifically, on a natural code base [16]. We read only their titles here. Most are trained on C/C++ or on token sequences and none returns a reviewer-facing explanation and patch.

Techniques reused. SMOTE [6] creates minority samples by interpolating between a sample and its nearest minority neighbours, Tomek links [7] are pairs of nearest neighbours from different classes whose majority member is removed, and Batista et al. [8] compare their combination with other balancing methods. Random Forest [9] and gradient-boosted trees [4] are the standard learners for tabular metric data; they are cheap to run on a CPU, which matters when a model must score every function of every pull request. Repository retrieval builds on TF-IDF weighting [17] and on link analysis over a graph [18].

TABLE I
RELATED WORK

Work	Approach	Data	Reported result	Limitation
[1] base paper	14 CPG metrics, 7 selectors, SmoteTomek, 9 models	21,759 C/C++ functions, 275 vulnerable	RF acc 0.96, F1 0.961	C/C++; offline; no explanation
Sphinx	Checklist benchmark, rule-reward training	PR review data	Up to 40 % more checklist coverage	Comments only; data not yet released
CNN-LSTM; AI audit	Hybrid network; detect and remediate	Not retrieved	Not retrieved	Titles only
Multi-modal; DcodeBERT	Meta-learning; detect and repair	Not retrieved	Not retrieved	Titles only
Deep / AST learners	AST, deep, Python code learning	Not read	Not read	Not read here

Gap. The metric signal has been used offline, review models produce comments without a security engine or code metrics, and we found no system that joins metric-based risk, taint rules, verified patches and repository-aware answers in the pull-request workflow, evaluated openly including its weaknesses. AutoSecAI addresses this combination.

III. METHODOLOGY

A. Architecture and Pipeline

A React single-page application talks to a FastAPI service that wraps the analysis engine (Fig. 1). Long operations run as background jobs whose agent messages the interface polls, so the user watches each agent finish. Accounts (bcrypt, JWT) live in one SQLite file and saved reviews, stored as JSON, in another. A review passes through seven agents (Fig. 2).

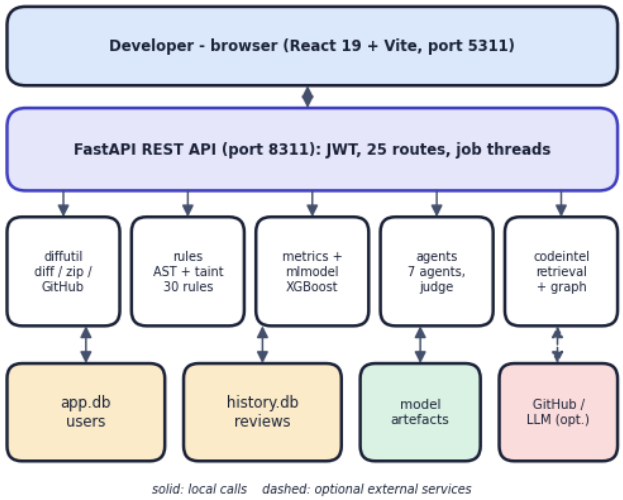


Fig. 1. System architecture of AutoSecAI.

B. Metrics, Corpus and Risk Model

The Python ast module replaces the code property graph. Metrics M1-M11 keep the families of the base paper (cyclomatic complexity, size, variables, calls, parameters, index arithmetic in place of pointer arithmetic, I/O calls, nesting, loops, control dependency). Because Python flaws are injections rather than memory errors, three taint-oriented metrics are added: M12 risky API calls, M13 string-built sinks and M14 untrusted or secret-like sources. The C datasets are not distributable, so a corpus of 6,000 Python

functions is generated: random benign logic of varying complexity, injected patterns of eight types (SQL and command injection, code execution, deserialisation, path traversal, XSS, hard-coded secret, weak cryptography), look-alike safe code as hard negatives, and 1 % label noise. 501 functions (8.35 %) are vulnerable. Metrics are measured on the real generated source.

The metric extractor visits each function once. Cyclomatic complexity counts decision points and boolean operators plus one; nesting and control dependency are the deepest block depth and the largest number of guarding conditions on a statement; M12 counts calls from a list of dangerous APIs (eval, exec, os.system, subprocess, pickle and yaml loaders, weak hashes and random generators); M13 counts call arguments built by f-string, concatenation, percent or format; and M14 counts request-like sources and secret-named literals. Each metric is a plain count, so a reviewer can see why a function scored high.

Seven selectors (Pearson, linear SVM, Lasso, chi-squared, two RFE variants, variance) vote on the training split. The paper keeps five metrics; here the number k is validated from 5 to 14 and k = 13 gives the best validation F1. Values are Min-Max scaled and the minority class is balanced by SMOTE [6] followed by Tomek-link removal [7], implemented in house: a synthetic sample is given by Eq. (1), with λ drawn uniformly between 0 and 1 and x_nn one of the five nearest minority neighbours, and a Tomek link is removed by dropping its majority member. Nine models are trained on 8,232 balanced rows and tested on 1,500 untouched functions: Random Forest (300 trees), decision tree, logistic regression, RBF SVM, KNN, naive Bayes, gradient boosting, XGBoost and a soft-voting ensemble. The best F1 model, XGBoost, is stored. A second Random Forest classifies the vulnerability type from all 14 metrics.

$$x_{new} = x + \lambda\,(x_{nn} - x), \quad \lambda \sim U(0, 1) \tag{1}$$

Quality is reported as precision P = TP / (TP + FP), recall R = TP / (TP + FN) and their harmonic mean F1 = 2PR / (P + R), because with 8.35 % positives accuracy alone would reward a model that flags nothing.

C. Rule Engine and Agents

The static agent walks the AST with taint tracking: values from request objects, input, sys.argv and the environment are sources, calls such as cursor.execute, os.system, eval, pickle.loads and file opens are sinks, and sanitisers (parameter binding, shlex, escaping) stop the flow. It holds 30 security rules, each with a CWE [19] and OWASP class, severity, impact, remediation and a safe pattern, plus 12 quality rules; JavaScript, PHP and Java get regex rules. The ML agent scores each changed function and reports its three largest drivers. The reasoner merges both signals and marks a finding as corroborated when the model gives $p \geq 0.5$. The remediation agent rewrites the code (for example parameterising a query), re-parses it and re-runs the rules, and keeps the patch only if the finding is gone.

Every agent appends an entry to a trace (name, role, seconds, summary, details) that is stored with the review, so the interface can show what each agent did. Findings carry the rule, CWE and OWASP class, severity, confidence, the code line and a source-to-sink description; findings on lines the pull request did not touch are kept but marked as pre-existing and weighted down, so a reviewer is not blamed for old code. A minimum-severity filter and an ML on/off switch are options of the review.

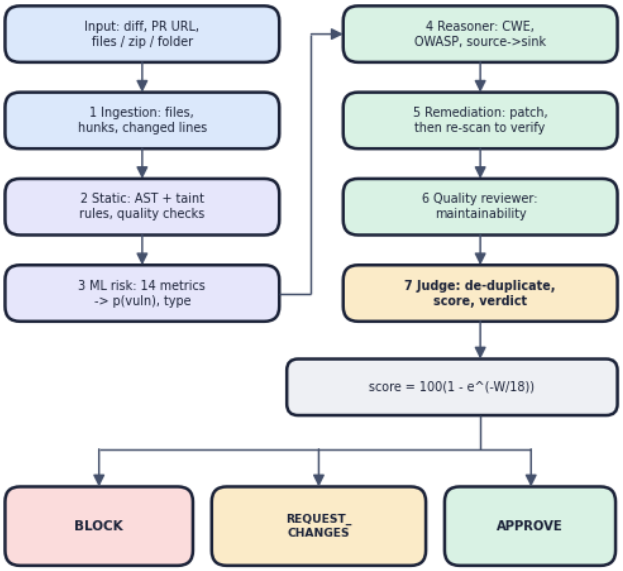


Fig. 2. Review flow of the seven agents and the verdict.

The judge removes duplicates with the same file, line and CWE, and gives a finding the weight $w = s \cdot \kappa \cdot \gamma$, where s is the severity weight (critical 10, high 6, medium 3, low 1, info 0.2), κ the confidence weight (1.0, 0.7, 0.4) and $\gamma = 1$ inside the changed lines and 0.15 outside. With W the sum of the weights, the risk score is

$$score = round(100 \cdot (1 - exp(-W / 18))) \tag{2}$$

The verdict is BLOCK if a changed high or critical security finding has medium or high confidence, or the score is at least 70; REQUEST_CHANGES if a changed medium finding exists or the score is at least 30; otherwise APPROVE.

D. Codebase Assistant

A repository is cut into function, class and window chunks. Retrieval scores a chunk with TF-IDF [17] over split identifiers (unigrams and bigrams), adds 0.30 times a concept affinity for the detected topic (authentication, routing, database, user interface, configuration, tests, security), boosts exact identifier and path matches, and adds a personalised PageRank [18] boost ($\alpha = 0.6$, weight 0.08) over the import and call graph. The answer lists the evidence chunks with the reasons they matched; an LLM may rewrite the text only if a key is given.

E. Implementation

The service has 25 route handlers (7 public) with HS256 tokens valid for 12 h and bcrypt with 10 rounds. Reviews accept a diff, a before and after pair, a GitHub pull-request URL, or files, a zip or a folder, and can be exported as

Markdown, SARIF 2.1.0 [20], JSON or CSV. The engine has 2,881 non-blank lines, the service 502 and the React interface 6,525.

Inputs are validated before work starts: an empty or malformed diff, an unreadable folder or an unreachable GitHub address is rejected with a readable message, uploads are limited to 300 files per review and 800 KB per file, and passwords need eight characters. Jobs are held in memory and pruned after an hour; at most six repository indexes are kept. The tool never posts a comment or changes a repository: every output is a report that a person reads, and the history of reviews can be deleted from the interface.

IV. RESULTS

A. Test Conditions

Models are evaluated on a stratified 25 % split (1,500 functions, 125 vulnerable) of the generated corpus, with the training part balanced and the test part untouched. To check stability the pipeline is repeated on five independently generated corpora (k fixed at 13). The rule engine is scored on the same 1,500 functions: a function counts as flagged if it has a security finding of medium severity or higher. Latency is the median of 20 runs on the demo pull request (one file, +67/−1 lines, 7 functions) after a warm-up. Codebase retrieval is checked with 12 hand-written questions on the built-in demo repository (10 files). All numbers are measured on the machine of Section I-A.

TABLE II
NINE MODELS ON 1,500 HELD-OUT FUNCTIONS

Model	Acc.	Prec.	Rec.	F1	AUC
Random Forest	0.973	0.882	0.776	0.826	0.953
Decision Tree	0.953	0.682	0.824	0.746	0.896
Logistic Regression	0.879	0.401	0.920	0.558	0.945
SVM	0.933	0.563	0.856	0.679	0.929
KNN	0.901	0.448	0.832	0.583	0.924
Naive Bayes	0.877	0.397	0.920	0.554	0.936
Gradient Boosting	0.949	0.650	0.832	0.730	0.937
XGBoost	0.977	0.942	0.776	0.851	0.937
Voting Ensemble	0.963	0.738	0.856	0.793	0.956

B. Risk Model

XGBoost gives the best F1 (0.851, precision 0.942, recall 0.776) and Random Forest the highest AUC of the single models (0.953); only 6 safe functions are flagged and 28 vulnerable ones missed. Over five corpora XGBoost reaches F1 0.842 ± 0.006 (range 0.836-0.851) and Random Forest 0.814 ± 0.010, so the headline value is the best of five. With the paper's five metrics and SMOTE-Tomek the Random Forest F1 falls to 0.645, against 0.847 with all 14 metrics and no balancing and 0.826 with 13 metrics and balancing. The settings differ in both metric count and balancing, so the effect of balancing alone is not isolated; recall is identical (0.776) with and without it. The type classifier reaches 0.651 accuracy over eight types: SQL injection (F1 0.875) and XSS (0.872) are recognised, deserialisation (0.207) and path traversal (0.333) are not.

The metrics that matter most are those that were added for Python: in the Random Forest of the Model Lab (the shipped scorer is XGBoost) the largest importances are risky API calls (0.174), untrusted sources (0.147) and string-built sinks (0.143); the three together carry about 46 % of the total, whereas cyclomatic complexity carries 0.045. The base paper's complexity-led ranking therefore does not transfer to injection flaws in Python.

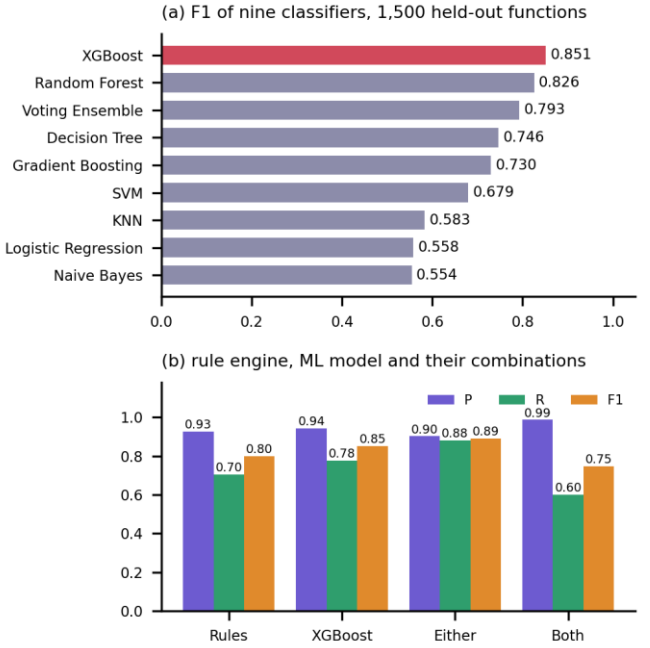


Fig. 3. F1 of nine models (a); rule engine, ML model and their combinations (b).

C. Rule Engine, ML Model and Their Combination

On the same functions the rule engine reaches precision 0.926, recall 0.704 and F1 0.800; the XGBoost model 0.851. Flagging a function when either detector does reaches recall 0.880 and F1 0.891 (Fig. 3b), while requiring both gives precision 0.987 at recall 0.600. The detectors are complementary, which supports using both in one pipeline. The rule figures are optimistic: the generator and the rules were written by the same team around the same eight patterns, and 1 % label noise creates positives without vulnerable code.

In counts, XGBoost misses 28 of 125 vulnerable functions and flags 6 of 1,375 safe ones; the rule engine finds 88, misses 37 and flags 7; the union finds 110, misses 15 and flags 12. For a reviewer the trade-off is visible: the union catches 13 more vulnerable functions than XGBoost for 6 more false alarms, while requiring agreement is the setting for a gate that must almost never block a safe change. In the product both signals reach the judge, which weighs the findings rather than trusting either detector alone.

D. Demonstration, Speed and Tests

The built-in demo pull request (67 added lines) receives score 99 and verdict BLOCK: 17 findings (1 critical, 10 high, 4 medium, 2 info), all 7 functions flagged and all 7 generated patches verified by re-scan (Figs. 4 and 5). A review takes a median of 0.375 s (95th percentile 0.471 s); the ML agent needs 0.30 s of it. Over HTTP a polled review job finishes in a median of 436 ms and a login, dominated by bcrypt, takes 60 ms. The assistant ranks the expected file first for 8 of 12

questions and in the top three for 11; an index of 10 files builds in 14 ms. Seventeen pytest tests, 16 API smoke checks and 22 browser flows pass with no console errors.

The 15 changed-line security findings span 12 weakness classes: two each for CWE-798 (hard-coded credential), CWE-89 (SQL injection) and CWE-502 (unsafe deserialisation), and one each for CWE-22, 78, 79, 95, 295, 327, 338, 489 and 918. The static agent needs 8 ms, remediation 62 ms and the ML agent 302 ms, so scoring the functions through the stored model dominates a review. The question the assistant missed asked how a user is looked up by name; it returned the authentication module and a test file instead of the database module that defines the lookup. These measurements come from one idle machine and one small demonstration; they show that the pipeline is fast enough to run on every pull request, not how it scales to large changes.

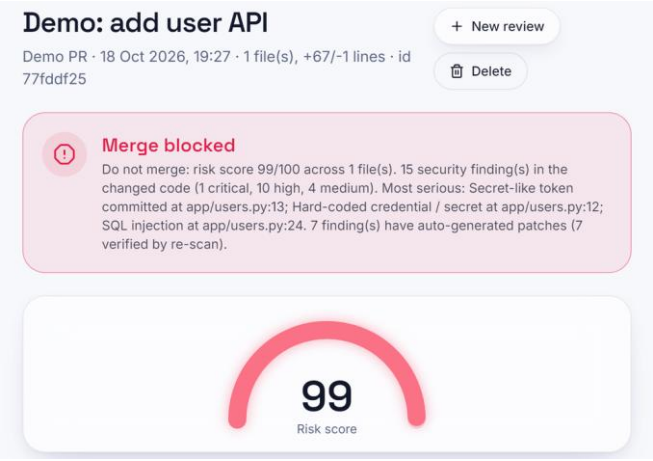


Fig. 4. Verdict and risk score of the demo pull request.

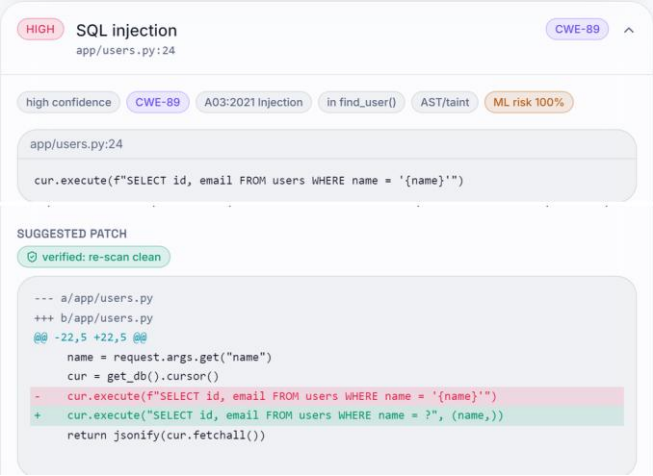


Fig. 5. A finding with its verified patch.

E. Comparison and Limits

The base paper [1] reports F1 0.961 for Random Forest on real C/C++ functions; this work reports 0.826 for its Random Forest and 0.851 for the XGBoost model it ships, on generated Python functions. The language, data, class balance and parser differ, so the numbers are not comparable and no superiority is claimed; the contribution is the integration. Failure cases and limits: the corpus is synthetic, so scores are a prioritisation signal rather than proof of a flaw; the rule engine misses 37 of 125 positives; the type classifier is weak for three types; retrieval ranked the right file first for only 8 of 12 questions (a lookup question missed it entirely); the whole-file fix needs full-file input; saved reviews are not separated by account; no comparison with Bandit, Semgrep or CodeQL was run; precision against manually reviewed real pull requests, objective 6 of the project, was not measured; and the optional LLM explanations were not evaluated.

All figures can be regenerated: the corpus, models and report come from one training call of about 32 s, the measurement script writes the engine, seed, latency and retrieval numbers, and the screens in Figs. 4 and 5 were captured from the running application with real data. The 17 automated tests check the engine (SQL injection found and safe code not flagged, taint from a request, sanitiser stopping taint, auto-fix that re-parses, cyclomatic metric, diff parsing, pre-existing findings, ML ranking, SMOTE-Tomek balance, repository retrieval) and the service (token checks, demo login, registration rules, a full review with SARIF export and deletion, input validation, codebase questions, the Model Lab scorer).

V. CONCLUSION

AutoSecAI brings metric-based risk, taint rules, verified patches and repository-aware answers into one pull-request review. On generated Python data XGBoost reaches F1 0.851 (mean 0.842 over five corpora), the rule engine 0.800, and their union 0.891; the demo review runs in 0.38 s. Against the objectives, changed files and diff context are extracted, metrics rank functions, AST rules detect injection flaws, agents cover security and quality, retrieval supplies repository context, and a consolidated verdict is produced; the precision test on real reviewed pull requests remains open. The main finding is that the two detectors are complementary. The practical value is a reviewer-facing report that is explained, patched and checked in under half a second for a small change. Future work: a labelled set of real pull requests and a comparison with existing analysers such as Bandit, Semgrep and CodeQL; real Python vulnerability data for the model, with a per-language corpus; a measured evaluation of the language-model layer and of embedding-based retrieval; and per-account storage with CI and webhook integration so that a verdict can gate a merge.

REFERENCES

[1] B. K. E. Abbasi and A. Shameli-Sendi, "Leveraging key source code metrics for predicting software vulnerabilities with machine learning," IEEE Access, vol. 14, pp. 52610-52625, 2026, doi: 10.1109/ACCESS.2026.3679930.

[2] D. Zhang, S. Zhang, Z. Jin, J. Luo, S. Fu and E. Nallipogu, "Sphinx: Benchmarking and modeling for LLM-driven pull request review," arXiv:2601.04252, Jan. 2026.

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, 2016, pp. 785-794.

[5] T. J. McCabe, "A complexity measure," IEEE Trans. Softw. Eng., vol. SE-2, no. 4, pp. 308-320, Dec. 1976.

[6] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321-357, 2002.

[7] I. Tomek, "Two modifications of CNN," IEEE Trans. Syst., Man, Cybern., vol. SMC-6, no. 11, pp. 769-772, Nov. 1976.

[8] G. E. A. P. A. Batista, R. C. Prati and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," ACM SIGKDD Explor. Newsl., vol. 6, no. 1, pp. 20-29, 2004.

[9] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, 2001.

[10] "Enhancing software security with CNN-LSTM hybrid model: Vulnerability detection," 2025 (listed in the batch literature survey; not independently retrieved).

[11] "Real-time AI code security auditing: Automated vulnerability detection and remediation through meta-experimental analysis," 2025 (listed in the batch literature survey; not independently retrieved).

[12] "Improving software security: A multi-modal framework for vulnerability detection using code simplification and meta-learning," 2025 (listed in the batch literature survey; not independently retrieved).

[13] "Advancing software security: DcodeBERT for automatic vulnerability detection and repair," 2024 (listed in the batch literature survey; not independently retrieved).

[14] F. Yamaguchi, M. Lottmann and K. Rieck, "Generalized vulnerability extrapolation using abstract syntax trees," in Proc. 28th Annu. Comput. Secur. Appl. Conf., Dec. 2012, pp. 359-368.

[15] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng and Y. Zhong, "VulDeePecker: A deep learning-based system for vulnerability detection," arXiv:1801.01681, 2018.

[16] L. Wartschinski, Y. Noller, T. Vogel, T. Kehrer and L. Grunske, "VUDENC: Vulnerability detection with deep learning on a natural codebase for Python," Inf. Softw. Technol., vol. 144, Art. no. 106809, Apr. 2022.

[17] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Inf. Process. Manage., vol. 24, no. 5, pp. 513-523, 1988.

[18] L. Page, S. Brin, R. Motwani and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Stanford, CA, USA, Tech. Rep. 1999-66, 1999.

[19] MITRE Corporation, "Common Weakness Enumeration (CWE)." [Online]. Available: https://cwe.mitre.org/

[20] OASIS, "Static Analysis Results Interchange Format (SARIF) Version 2.1.0," OASIS Standard, Mar. 2020.